

## Chapter 6

# Performance considerations

GPU memory behavior, latency hiding, and common CUDA optimization patterns

Main idea: performance is about matching the kernel's resource demands to the hardware's resource constraints.

# What this chapter adds

Context from Chapters 1-5

- Global memory access coalescing
- Hiding DRAM latency with many requests in flight
- Vector loads and stores
- Shared memory bank conflicts
- Thread coarsening, loop unrolling, and double buffering
- A checklist for deciding what to optimize next

Optimization is not a fixed recipe. Each technique trades one resource for another.

Example tradeoff: use shared memory to reduce global memory traffic, but this may reduce occupancy.

Example tradeoff: coarsen threads to reduce overhead, but too much coarsening can underutilize the GPU.

# Performance tuning starts with bottlenecks

## High-level strategy

- A kernel is limited by the resource it stresses most
- Useful optimizations reduce pressure on the bottleneck resource
- Bad optimizations add pressure to the bottleneck resource
- The process is iterative: profile, change, profile again

Question to keep asking:  
What is the bottleneck, and why?

Compute cores idle? Look at occupancy, divergence, instruction stalls.

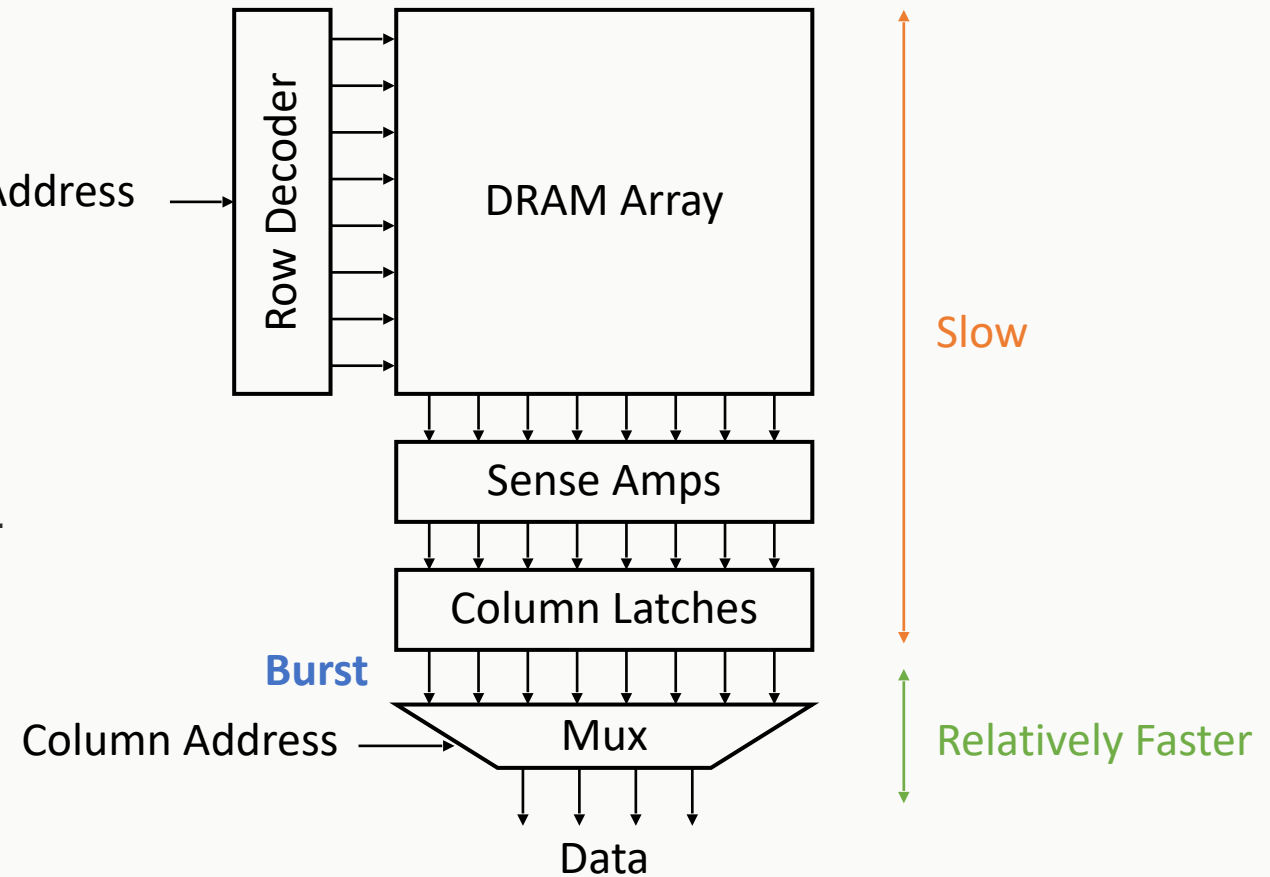
Memory system overloaded? Look at traffic, coalescing, reuse, bank conflicts.

Threads waiting at barriers? Look at synchronization and false dependences.

# DRAM: why global memory is slow

DRAM background from the PPT

- DRAM cells store bits as charge in tiny capacitors
- A row decoder activates many cells together
- Sense amplifiers detect weak charge changes
- Latency is high, so DRAM relies on parallelism for throughput



# Bursts and cache lines

Why adjacency matters

| Address |   |   |
|---------|---|---|
| 0       | 1 | 0 |

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

- A DRAM access returns a range of consecutive locations
- This range is a DRAM burst
- Caches also move data in cache-line sized chunks
- Using most bytes from each burst or cache line improves effective bandwidth

Bad case: random addresses across many bursts.



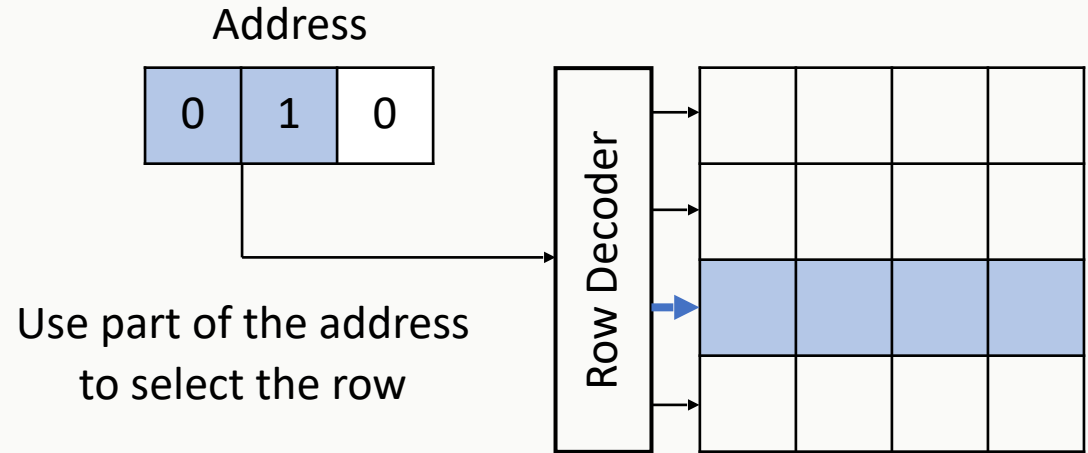
Good case: nearby addresses in the same burst or cache line.



# Bursts and cache lines

Why adjacency matters

- A DRAM access returns a range of consecutive locations
- This range is a DRAM burst
- Caches also move data in cache-line sized chunks
- Using most bytes from each burst or cache line improves effective bandwidth



Bad case: random addresses across many bursts.

Timeline:

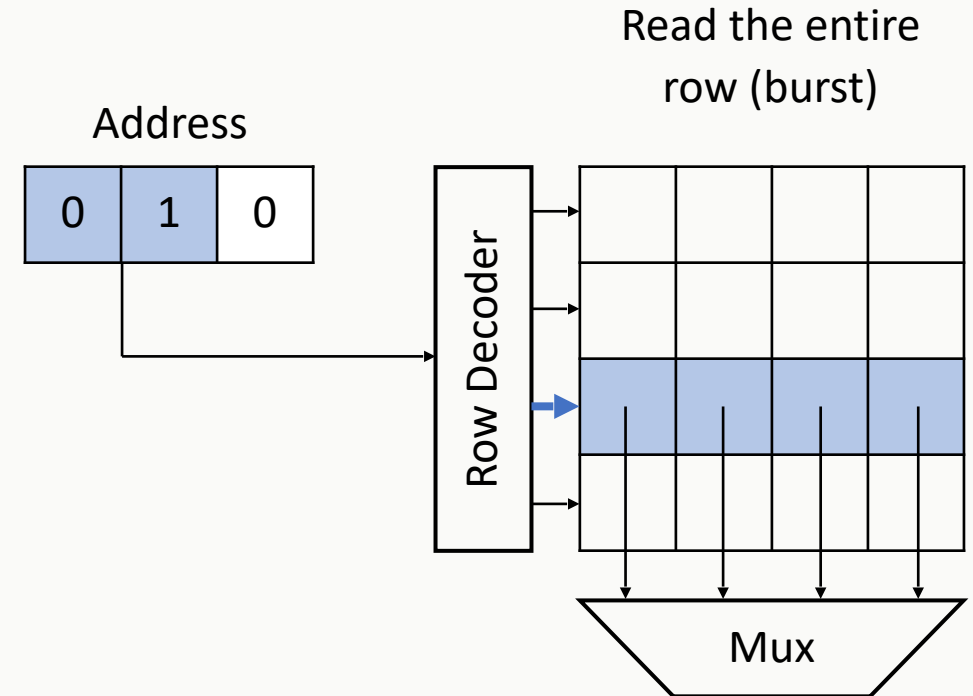
Good case: nearby addresses in the same burst or cache line.

Timeline:

# Bursts and cache lines

Why adjacency matters

- A DRAM access returns a range of consecutive locations
- This range is a DRAM burst
- Caches also move data in cache-line sized chunks
- Using most bytes from each burst or cache line improves effective bandwidth



Bad case: random addresses across many bursts.



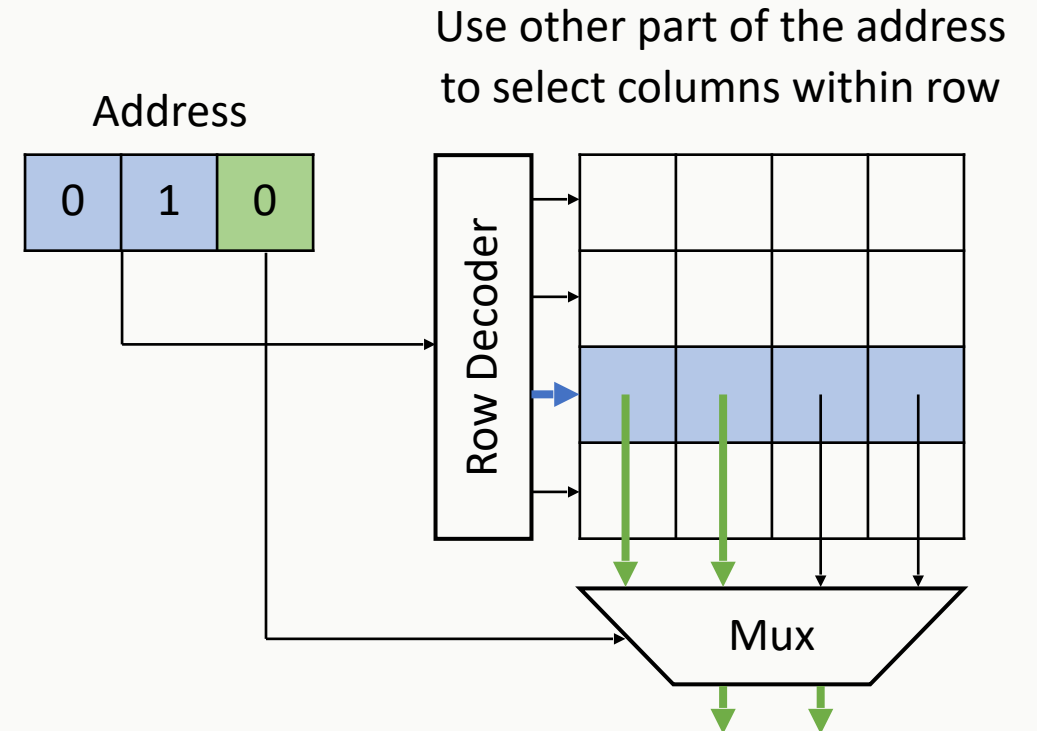
Good case: nearby addresses in the same burst or cache line.



# Bursts and cache lines

Why adjacency matters

- A DRAM access returns a range of consecutive locations
- This range is a DRAM burst
- Caches also move data in cache-line sized chunks
- Using most bytes from each burst or cache line improves effective bandwidth



Bad case: random addresses across many bursts.

Timeline: 

The timeline diagram for the 'Bad case' shows a sequence of memory accesses over time. It consists of a long orange bar, followed by a short green bar, followed by another long orange bar, and finally a short green bar. This represents a pattern where accesses are spread across many different bursts (orange) and cache lines (green).

Good case: nearby addresses in the same burst or cache line.

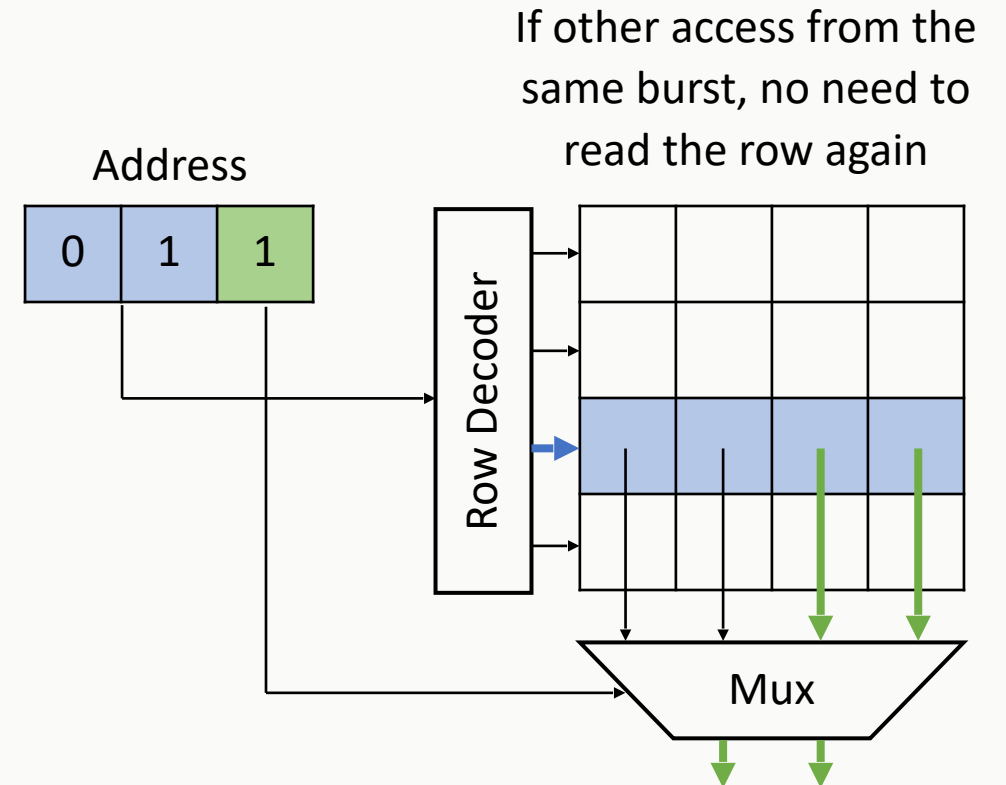
Timeline: 

The timeline diagram for the 'Good case' shows a sequence of memory accesses over time. It consists of a long orange bar, followed by two short green bars. This represents a pattern where accesses are concentrated within a few bursts (orange) and cache lines (green), leading to better utilization.

# Bursts and cache lines

Why adjacency matters

- A DRAM access returns a range of consecutive locations
- This range is a DRAM burst
- Caches also move data in cache-line sized chunks
- Using most bytes from each burst or cache line improves effective bandwidth



Bad case: random addresses across many bursts.

Timeline: 

Good case: nearby addresses in the same burst or cache line.

Timeline: 

# Memory coalescing

## Section 6.1

- Threads in a warp execute the same load or store instruction together
- If they access consecutive global memory locations, hardware combines the accesses
- Best case: consecutive and aligned within burst or cache line boundaries
- Result: fewer memory transactions and less global memory traffic

Coalescing is like carpooling for memory requests: fewer vehicles on the same road.

Important: coalescing applies to global memory loads and stores.

Uncoalesced stores can be especially expensive because of partial store traffic and read-modify-write behavior.

# Row-major layout controls adjacency

## Section 6.1

- C and CUDA arrays use row-major layout
- Adjacent elements in a row are adjacent in memory
- Adjacent elements in a column are separated by the row width
- Coalescing depends on physical addresses, not just the matrix picture

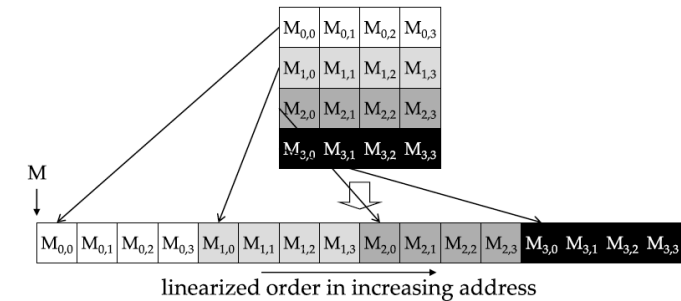
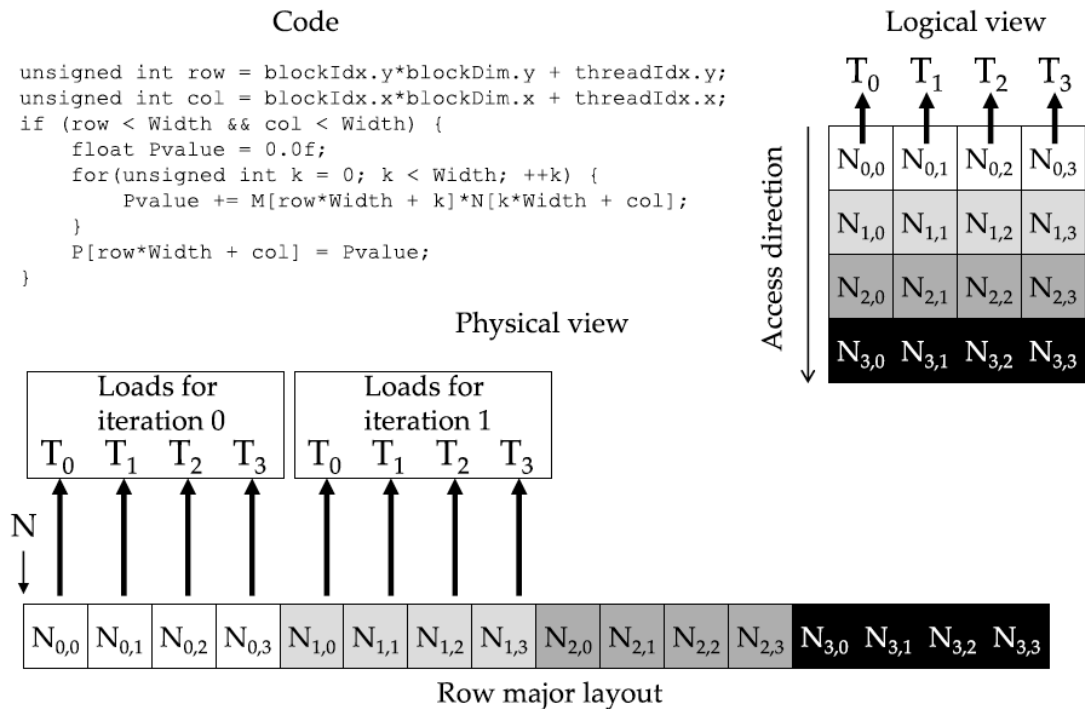


FIGURE 6.1

Placing matrix elements into a linear array based on row major order.

For  $M[\text{row}][\text{col}]$ , linear index =  $\text{row} * \text{Width} + \text{col}$

# Coalesced matrix access



- Consecutive threads have consecutive col values
- $N[k*Width + col]$  becomes consecutive addresses
- Each iteration can be served efficiently by coalesced memory requests

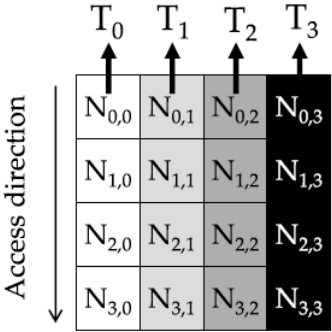
**FIGURE 6.2**  
A coalesced access pattern.

# Uncoalesced matrix access

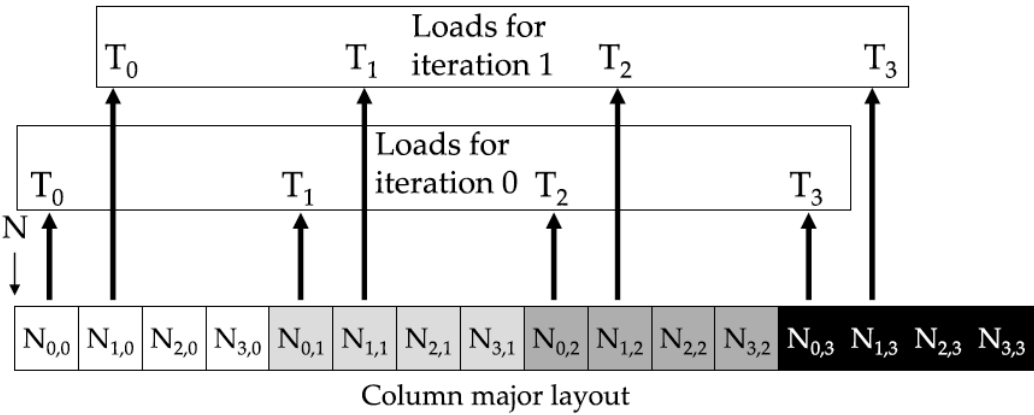
### Code

```
unsigned int row = blockIdx.y*blockDim.y + threadIdx.y;
unsigned int col = blockIdx.x*blockDim.x + threadIdx.x;
if (row < Width && col < Width) {
    float Pvalue = 0.0f;
    for(unsigned int k = 0; k < Width; ++k) {
        Pvalue += M[row*Width + k]*N[col*Width + k];
    }
    P[row*Width + col] = Pvalue;
}
```

### Logical view



### Physical view

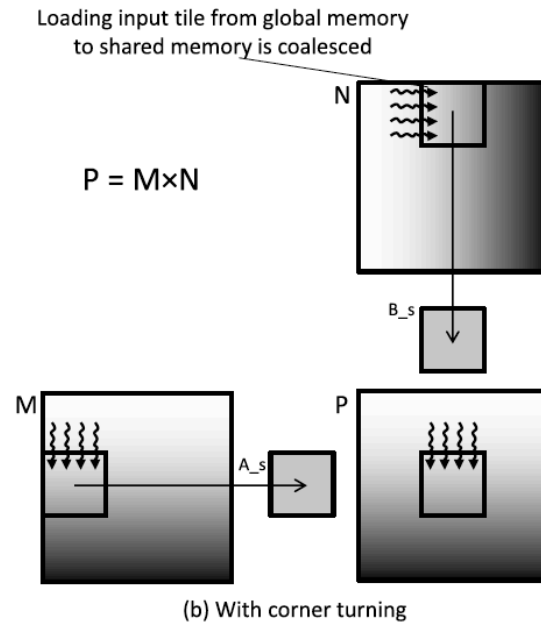
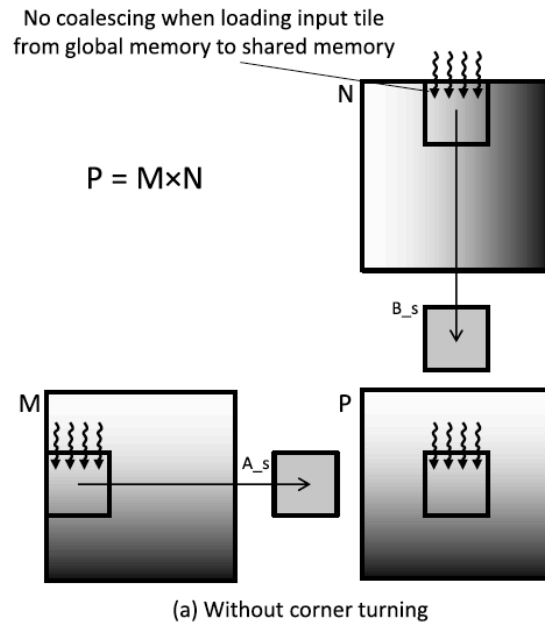


- Logical access may still look regular
- Physical addresses are no longer adjacent
- The warp generates many memory transactions instead of a consolidated one

**FIGURE 6.3**

An un-coalesced access pattern.

# Corner turning



- Problem: input matrix N is effectively column-major
- Naive loading gives strided global memory accesses
- Fix: map consecutive threads to consecutive global addresses
- Store the tile into shared memory in the layout needed by the computation

**FIGURE 6.4**

Applying corner turning to coalesce accesses to matrix N which is stored in column-major layout.

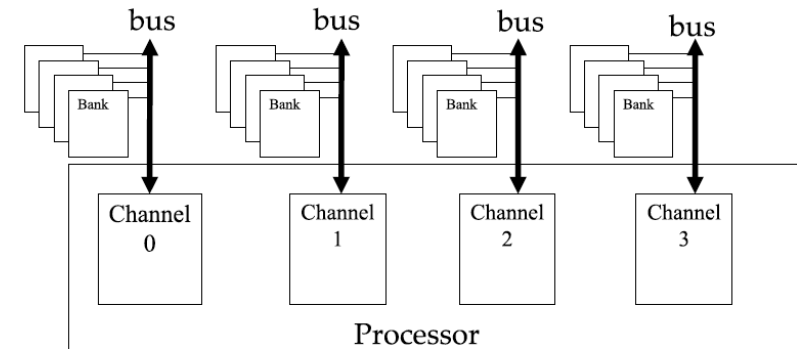
Key pattern: coalesced global load, flexible shared memory access.

# **Coalescing: NVIDIA Technical Blog, Unlock GPU Performance: Global Memory Access in CUDA, 2025**

# DRAM channels and banks

Section 6.2 and Fig. 6.7

- Bursts alone do not provide enough bandwidth
- DRAM systems add channels and banks
- A channel has a bus connecting banks to the processor
- Multiple banks let access latency overlap with data transfer



**FIGURE 6.7**

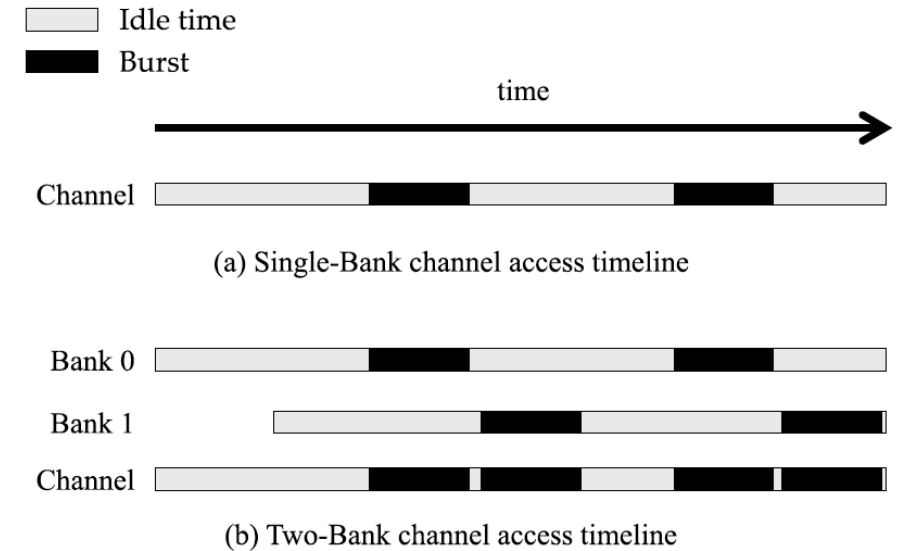
Channels and banks in DRAM systems.

The memory system is parallel. The kernel must generate enough independent memory requests to use it.

# Banking hides DRAM latency

Fig. 6.8

- A single bank leaves the channel bus idle during cell-array access
- With multiple banks, one bank can transfer while others prepare data
- If latency / transfer time =  $R$ , roughly  $R + 1$  banks are needed to saturate the bus
- Conflicts happen when simultaneous accesses target the same bank



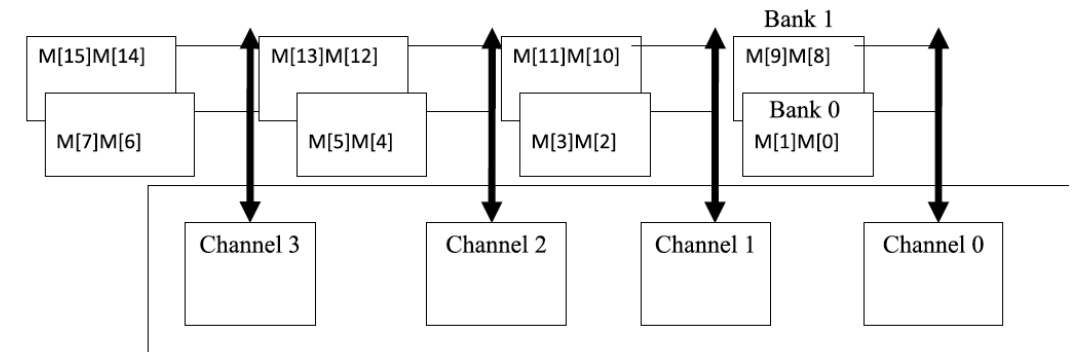
**FIGURE 6.8**

Banking improves the utilization of data transfer bandwidth of a channel.

# Interleaving across channels and banks

Fig. 6.9

- Array elements are distributed by hardware address mapping
- The layout interleaves across channels and banks
- Goal: nearby regions can use multiple memory resources
- Enough simultaneous requests are needed to benefit from this organization



**FIGURE 6.9**

Distributing array elements into channels and banks.

# Occupancy also helps memory bandwidth

## Section 6.2

- High occupancy gives the SM more warps to schedule
- It also gives the memory system more requests in flight
- This helps hide DRAM access latency
- But bandwidth utilization efficiency still depends on coalescing and bank/channel distribution

Occupancy is not the final goal. It is useful only if the extra warps help hide the current bottleneck.

Too many resources per block can reduce occupancy.

Too little work in flight can underutilize memory bandwidth.

Profiling can help decide which one matters.

# Vector loads and stores

## Section 6.3

- Each thread can load or store a larger continuous chunk
- float4 represents four 32-bit floats, 16 bytes total
- Fewer load/store instructions for the same data volume
- Especially useful when occupancy is not high enough to issue many scalar accesses

```
01  __global__ void vecadd_kernel(float* x, float* y, float* z, int N) {  
02      int i = blockDim.x*blockIdx.x + threadIdx.x;  
03      float4 x4 = ((float4*)x)[i];  
04      float4 y4 = ((float4*)y)[i];  
05      float4 z4;  
06      z4.x = x4.x + y4.x;  
07      z4.y = x4.y + y4.y;  
08      z4.z = x4.z + y4.z;  
09      z4.w = x4.w + y4.w;  
10      ((float4*)z)[i] = z4;  
11  }
```

**FIGURE 6.12**

A vector addition kernel that uses vector loads.

Caution: handle n not divisible by 4 with scalar cleanup or boundary checks.

Caution: alignment matters for efficient vectorized access.

# Shared memory bank conflicts

## Section 6.4

- Shared memory is SRAM, not DRAM
- Typical organization: 32 banks, each 4 bytes wide
- Consecutive 4-byte words map to consecutive banks
- If a warp accesses the same bank many times, accesses are serialized

Bank number for 4-byte words:  $\text{index} \bmod 32$

Best case: 32 threads touch 32 different banks.

Worst case: all 32 threads touch the same bank, a 32-way bank conflict.

## **Bank Conflict: NVIDIA Technical Blog, An Efficient Matrix Transpose in CUDA C/C++, Mark Harris, 2013**

# Thread coarsening

## Section 6.5

- Fine-grained mapping: one small unit of work per thread
- Coarsening: each thread handles multiple units of work
- Goal: reduce the price of parallelizing too much
- Examples: fewer redundant loads, fewer barriers, fewer instructions, better vectorization

Main risk: too much coarsening hides parallelism from the hardware.

Also watch register use, shared memory use, occupancy, and tail effects.

# Coarsening loop pattern

## Section 6.5 code

```
// Fine-grain version
unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
foo(i);

// Coarsened by a factor of 4
unsigned int iStart = 4 * (blockIdx.x * blockDim.x + threadIdx.x);

for (unsigned int c = 0; c < 4; ++c) {
    unsigned int i = iStart + c;
    foo(i);
}

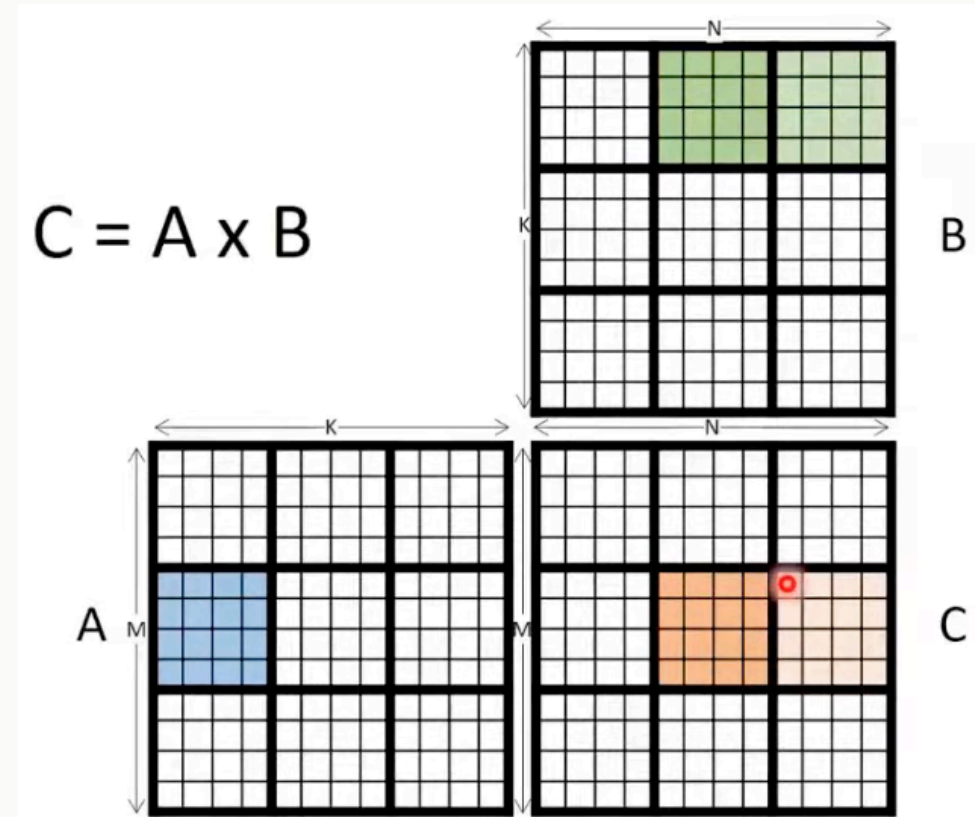
// Each thread processes four consecutive units of work
```

Pick the coarsening factor empirically. The best value is device-specific and data-size-specific.

# Coarsening tiled matrix multiplication

## Section 6.5

- Fine-grain tiled matmul: neighboring output tiles may load the same A tile
- Coarsened block computes multiple output tiles sequentially
- The A tile can be reused across several B tiles
- Benefit: less redundant global memory traffic



# Loop unrolling

## Section 6.6

- Replicate the loop body and reduce the number of iterations
- Fewer loop branches means fewer branch-related stalls
- Exposes independent instructions for scheduling
- Can turn variable local-array indices into constants
- This can allow register promotion after thread coarsening

```
for (unsigned int i = 0; i < 16; i += 4)
{
    A(i);      A(i + 1);
    A(i + 2);  A(i + 3);
    B(i);      B(i + 1);
    B(i + 2);  B(i + 3);
}
```

Compiler often unrolls profitable loops, but coarsening loops are especially important to think about.

# Double buffering

## Section 6.7

- Some barriers protect true dependences: read after write
- Some barriers protect false dependences: write after read on the same location
- Double buffering uses two buffers that swap roles
- Reads use inBuffer, writes use outBuffer
- This can remove one barrier per iteration

```
for (unsigned int i = 0; i < N; ++i) {  
    ... = buffer[anotherThreadID];  
    __syncthreads();  
    buffer[myThreadID] = ...;  
    __syncthreads();  
}
```

```
for (unsigned int i = 0; i < N; ++i) {  
    ... = inBuffer[anotherThreadID];  
    outBuffer[myThreadID] = ...;  
    __syncthreads();  
    swap(inBuffer, outBuffer);  
}
```

Cost: more memory. Benefit: less synchronization.

# Optimization checklist: already covered

Distributed view of Fig. 6.13

| Optimization                | Main benefit                       | Typical strategy                            |
|-----------------------------|------------------------------------|---------------------------------------------|
| Maximize occupancy          | hide core and memory latency       | tune blocks, registers, shared memory       |
| Coalesced global accesses   | reduce global memory traffic       | remap threads, remap layout, corner turning |
| Shared memory tiling        | reuse data within a block          | load once from global, reuse from shared    |
| Vector loads/stores         | reduce memory instruction overhead | load/store contiguous chunks                |
| Avoid shared bank conflicts | parallelize shared memory access   | padding or layout remapping                 |
| Thread coarsening           | reduce price of parallelism        | assign more work per thread                 |

# Optimization checklist: not covered deeply yet

Fig. 6.13

- Control divergence: keep threads in a warp on similar control paths
- Register tiling: reuse values from registers before writing back (kind of covered)
- Privatization: update private copies first, then merge into the public result
- Warp-level primitives: synchronize and exchange data within a warp without full block barriers

These show up more strongly in later patterns: histogram, reduction, prefix sum, filtering, sparse formats, and advanced matrix multiplication.

Fig. 6.13 is best used as a diagnostic checklist

# Optimization strategy

## Section 6.9

- Identify the bottleneck with profiling tools
- Choose an optimization that targets that bottleneck
- Check what new resource pressure the optimization creates
- Measure again on the target device
- Stop when performance is close to the hardware limit

Roofline reminder: arithmetic intensity helps decide whether a kernel is closer to memory-bound or compute-bound.

Same kernel, different GPU, different bottleneck. Retuning is normal.

# Takeaways

## Summary

- Global memory performance depends heavily on how warp addresses map to bursts and cache lines
- Latency hiding requires enough work in flight and good use of channels and banks
- Shared memory is fast, but its banks still matter
- Coarsening, unrolling, and double buffering reduce overheads, but can have a cost
- The right optimization is the one that targets the measured bottleneck

# Sources

For citations and follow-up

- [Elsevier companion site for PMPP 4th edition](#)
- [NVIDIA Technical Blog: Unlock GPU Performance: Global Memory Access in CUDA, 2025](#)
- [NVIDIA Technical Blog: An Efficient Matrix Transpose in CUDA C/C++, Mark Harris, 2013](#)